

Spartan6 - DSP48A1

Supervisor: Eng. Kareem Wassem

Created by: Abdelrahman Moamen Eltaweel

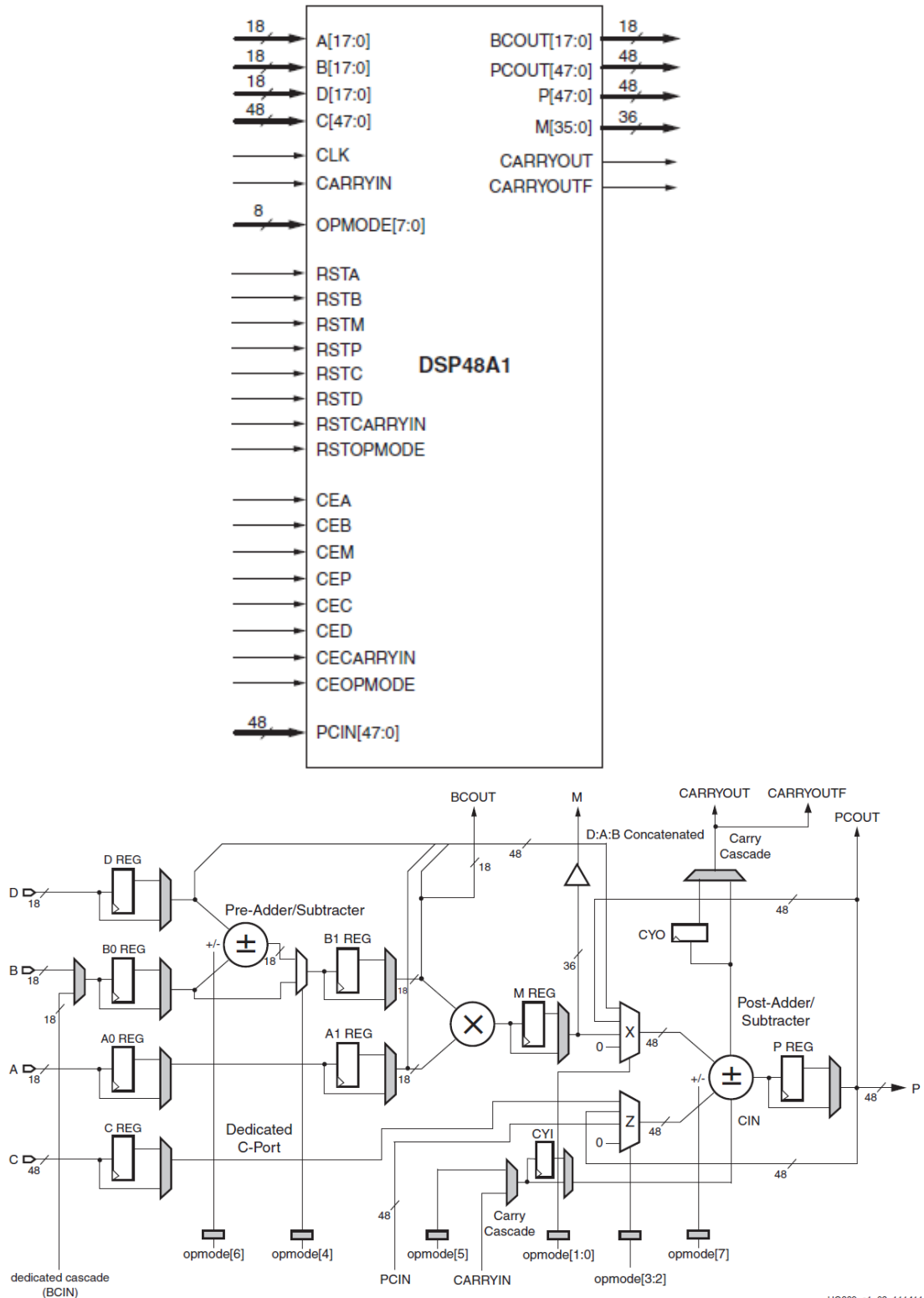
7/28/25

Table of Contents

DSP48A1 Slice Overview.....	4
Description	4
Key Features	4
Configuration and Usage	5
RTL code	5
Testbench code	11
Do file	17
QuestaSim Snippets	18
Verify Reset Operation	18
• Waveform	18
• Transcript	18
Verify DSP Path 1	19
• Waveform path 1	20
• Transcript path 1	20
Verify DSP Path 2	20
• Waveform path 2	21
• Transcript path 2	21
Verify DSP Path 3	22
• Waveform path 3	23
• Transcript path 3	23
Verify DSP Path 4	23
• Waveform path 4	24
• Transcript path 4	24
Constraint File	25
Elaboration.....	26
• Messages.....	26
• Schematic snippets	26
Synthesis	27

• Messages.....	27
• Utilization report.....	27
• Timing report	27
• Schematic snippets	28
Implementation	28
• Messages.....	28
• Utilization report.....	29
• Timing report	29
• Device snippets.....	29
Linting.....	30
• Snippets showing no errors	30

Spartan6 - DSP48A1



DSP48A1 Slice Overview

Description

The Spartan-6 FPGA series by Xilinx is designed for a wide range of applications that require low power, cost-effectiveness, and high performance. The DSP48A1 slice is an integral part of the Spartan-6 FPGA family, specifically optimized for digital signal processing (DSP) tasks. This slice is a flexible and powerful arithmetic unit that can perform various operations like addition, subtraction, multiplication, and more complex functions, making it a cornerstone for implementing DSP algorithms efficiently.

Key Features

16x16-bit Multiplier: Provides high-performance multiplication capability, essential for DSP algorithms.

48-bit Accumulator: Allows for large dynamic range in accumulation operations, critical for applications like filtering and FFTs.

Integrated Logic Functions: Supports various logical operations and shift functions, increasing the slice's versatility.

Pipelining and Bypass Options: Enables high-speed data processing with options for pipelining or bypassing stages, depending on the application requirements.

Low Power Consumption: Optimized for power efficiency, making it suitable for battery-operated and low-power designs.

Configurable Input and Output Registers: Flexible configuration options to support different operational modes and data paths.

Applications

Digital Signal Processing (DSP): Ideal for implementing high-performance DSP algorithms such as FIR filters, FFTs, and convolutions.

Wireless Communication: Used in modulation, demodulation, and other signal processing tasks in wireless communication systems.

Audio and Video Processing: Facilitates real-time audio and video processing, including encoding, decoding, and filtering.

Image Processing: Supports complex image processing operations like edge detection, filtering, and transformation.

Medical Electronics: Utilized in medical imaging devices and diagnostic tools for real-time data processing.

Configuration and Usage

The DSP48A1 slice can be configured in multiple modes to cater to various application needs. It can be used as a standalone arithmetic unit or as part of a larger, more complex DSP engine within the FPGA. Configuration involves setting up the slice through the Xilinx design tools (e.g., Vivado or ISE) to define the specific operations, pipelining, and data paths. The flexibility in configuration allows for tailored performance optimizations, enabling developers to balance power, speed, and resource utilization based on the application's requirements.

RTL code

```
module dsp48a1
(
    CLK, RSTA, RSTB, RSTC, RSTD, RSTM, RSTOPMODE, RSTP, RSTCARRYIN,
    CEA, CEB, CEC, CECARRYIN, CED, CEM, CEOPMODE, CEP,
    A, B, C, D, BCIN, PCIN, CARRYIN, OPMODE,
    CARRYOUT, CARRYOUTF, BCOUT, M, PCOUT, P
);
    ///! Parameter (Attributes):
    // The A0REG, A1REG, B0REG, and B1REG attributes can take values of 0 or 1.
    // these values define the number of pipeline registers in the A&B input paths.
    // A0REG defaults to 0 (no register). A1REG defaults to 1 (register). B0REG
    // defaults to 0 (no register) B1REG defaults to 1 (register).
    // A0 and B0 are the first stages of the pipelines.
    // A1 and B1 are the second stages of the pipelines
    parameter A0REG      = 1'b0;
    parameter A1REG      = 1'b1;
    parameter B0REG      = 1'b0;
    parameter B1REG      = 1'b1;

    // These attributes can take a value of 0 or 1.
    // The number defines the number of pipeline stages. Default: 1 (registered)
    parameter CREG       = 1'b1;
    parameter DREG       = 1'b1;
    parameter MREG       = 1'b1;
    parameter PREG       = 1'b1;
    parameter CARRYINREG = 1'b1;
    parameter CARRYOUTREG = 1'b1;
    parameter OPMODEREG  = 1'b1;
```

```

// The CARRYINSEL attribute is used in the carry cascade input, either the
CARRYIN input will be considered or the value of opcode[5].
// This attribute can be set to the string CARRYIN or OPMODE5.
// Default: OPMODE5. Tie the output of the mux to 0 if none of these string
values exist.
parameter CARRYINSEL = "OPMODE5";

// The B_INPUT attribute defines whether the input to the B port is routed from
the B input (attribute = DIRECT) or the cascaded input
// (BCIN) from the previous DSP48A1 slice (attribute = CASCADE).
// Default: DIRECT. Tie the output of the mux to 0 if none of these string values
exist.
parameter B_INPUT = "DIRECT";

// The RSTTYPE attribute selects whether all resets for the DSP48A1 slice should
have a synchronous or asynchronous reset capability.
// This attribute can be set to ASYNC or SYNC. Default: SYNC.
parameter RSTTYPE = "SYNC";
    // ! Data Ports:
input [17 : 0] A; //18-bit data input to multiplier, and optionally to postadder
subtractor depending on the value of OPMODE[1:0].
input [17 : 0] B; //18-bit data input to pre-adder/subtractor, to multiplier
depending on OPMODE[4], or to post-adder/subtractor depending
onOPMODE[1:0].
input [47 : 0] C; //48-bit data input to post-adder/subtractor.
input [17 : 0] D; //18-bit data input to pre-adder/subtractor. D[11:0] are
concatenated with A and B and optionally sent to post-
adder/subtractor depending on the value of OPMODE[1:0].
input [17 : 0] BCIN;
input CARRYIN; //carry input to the post-adder/subtractor
output [35 : 0] M; //36-bit buffered multiplier data output, routable to the FPGA
logic. It either the output of the M register (MREG = 1) or
the direct output of the multiplier (MREG = 0).
output [47 : 0] P; //Primary data output from the post-adder/subtractor. It is
either the output of the P register (PREG = 1) or the direct
output of the postadder/subtractor (PREG = 0).
output CARRYOUT; //Cascade carry out signal from post-adder/subtractor. It can
be registered in (CARRYOUTREG = 1) or unregistered
(CARRYOUTREG = 0). This output is to be connected only to
CARRYIN of adjacent DSP48A1 if multiple DSP blocks are used.
output CARRYOUTF; //Carry out signal from post-adder/subtractor for use in the
FPGA logic. It is a copy of the CARRYOUT signal that can be
routed to the user logic.

```

```

    ///! Control Input Ports
input CLK;           //DSP clock
input [7 : 0] OPMODE; //Control input to select the arithmetic operations of the
                      DSP48A1 slice.

    ///! Clock Enable Input Ports
input CEA;           //Clock enable for the A port registers: (A0REG & A1REG).
input CEB;           //Clock enable for the B port registers: (B0REG & B1REG).
input CEC;           //Clock enable for the C port registers (CREG).
input CECARRYIN;     //Clock enable for the carry-in register (CYI) and the carry-
                      out register (CYO).
input CED;           //Clock enable for the D port register (DREG).
input CEM;           //Clock enable for the multiplier register (MREG).
input CEOPMODE;      //Clock enable for the opmode register (OPMODEREG).
input CEP;           //Clock enable for the P output port registers (PREG = 1).

    ///! Reset Input Ports: All the resets are active high reset. They are either
    sync or async depending on the parameter RSTTYPE.
input RSTA;          //Reset for the A registers: (A0REG & A1REG).
input RSTB;          //Reset for the B registers: (B0REG & B1REG).
input RSTC;          //Reset for the C registers (CREG).
input RSTCARRYIN;    //Reset for the carry-in register (CYI) and the carry-out
                      register (CYO).
input RSTD;          //Reset for the D register (DREG).
input RSTM;          //Reset for the multiplier register (MREG).
input RSTOPMODE;     //Reset for the opmode register (OPMODEREG).
input RSTP;          //Reset for the P output registers (PREG = 1).

    ///! Cascade Ports:
output [17 : 0] BCOUT; //Cascade output for Port B.
input [47 : 0] PCIN;  //Cascade input for Port P.
output [47 : 0] PCOUT; //Cascade output for Port P.

    /// !first stage
    //register -> A0REG, B0REG, DREG, CREG, OPMODEREG
wire [17 : 0] a0, b0, d;    // A0REG, B0REG, DREG
wire [47 : 0] c;           //CREG
wire [7 : 0] opmode_reg;   //OPMODEREG

wire [17 : 0] b_select;     // register to select B or BCIN to store it in B0REG
                           based on the value of B_INPUT
assign b_select = (B_INPUT == "DIRECT") ? B : (B_INPUT == "CASCADE") ? BCIN
: 18'b0;

```

```

REG_MUX #(.sync_type(RSTTYPE), .WIDTH(8)) OPMODE_REG (CLK, RSTOPMODE, CEOPMODE,
OPMODEREG, OPMODE, opmode_reg);
REG_MUX #(.sync_type(RSTTYPE), .WIDTH(18)) A0_STAGE (CLK, RSTA, CEA, A0REG, A,
a0);
REG_MUX #(.sync_type(RSTTYPE), .WIDTH(18)) B0_STAGE (CLK, RSTB, CEB, B0REG,
b_select, b0);
REG_MUX #(.sync_type(RSTTYPE), .WIDTH(48)) C0_STAGE (CLK, RSTC, CEC, CREG, C, c);
REG_MUX #(.sync_type(RSTTYPE), .WIDTH(18)) D0_STAGE (CLK, RSTD, CED, DREG, D, d);

///! pre_adder/subtarcter
wire [17 : 0] out_pre;
assign out_pre = (opmode_reg[6] == 0) ? (d + b0) : (d - b0);

wire [17 : 0] out_mux_opmode_4;
assign out_mux_opmode_4 = (opmode_reg[4] == 1) ? out_pre : b0;

///! second stage
wire [17 : 0] a1, b1;
REG_MUX #(.sync_type(RSTTYPE), .WIDTH(18)) B1_STAGE (CLK, RSTB, CEB, B1REG,
out_mux_opmode_4, b1);
REG_MUX #(.sync_type(RSTTYPE), .WIDTH(18)) A1_STAGE (CLK, RSTA, CEA, A1REG, a0,
a1);
assign BCOUT = b1;

///! multiplication
wire [35 : 0] out_mul, mul_reg;
assign out_mul = b1 * a1;
REG_MUX #(.sync_type(RSTTYPE), .WIDTH(36)) mul (CLK, RSTM, CEM, MREG, out_mul,
mul_reg);
assign M = mul_reg;

///! x_multiplexer
wire [47 : 0] x_mux;
assign x_mux = (opmode_reg[1 : 0] == 2'b11) ? {d[11 : 0], a1[17 : 0], b1[17 : 0]}
: (opmode_reg[1 : 0] == 2'b10) ? PCOUT
: (opmode_reg[1 : 0] == 2'b01) ? {12'b0, mul_reg} : 0;

///! z_multiplexer
wire [47 : 0] z_mux;
assign z_mux = (opmode_reg[3 : 2] == 2'b11) ? c : (opmode_reg[3 : 2] == 2'b10) ?
PCOUT : (opmode_reg[3 : 2] == 2'b01) ? PCIN : 0;

```

```

//! carry in
wire carry_select;
wire cin;
assign carry_select = (CARRYINSEL == "OPMODE5") ? opmode_reg[5]
: (CARRYINSEL == "CARRYIN") ? CARRYIN : 1'b0;
REG_MUX #(.sync_type(RSTTYPE), .WIDTH(1)) CY1 (CLK, RSTCARRYIN, CECARRYIN,
CARRYINREG, carry_select, cin);

//! post_adder/subtractor
wire [47 : 0] out_post;
wire carry_out;
wire c_out;
assign {carry_out, out_post} = (opmode_reg[7] == 0) ? (z_mux + x_mux + cin)
: (z_mux - (x_mux + cin));

//! carry out
REG_MUX #(.sync_type(RSTTYPE), .WIDTH(1)) CY0 (CLK, RSTCARRYIN, CECARRYIN,
CARRYOUTREG, carry_out, c_out);
assign CARRYOUT = c_out;
assign CARRYOUTF = CARRYOUT;

//! output p
wire [47 : 0] out_p;
REG_MUX #(.sync_type(RSTTYPE), .WIDTH(48)) OUT_P_REG (CLK, RSTP, CEP, PREG,
out_post, out_p);
assign P = out_p;
assign PCOUT = P;
endmodule

//! The Pipeline_mux module
module REG_MUX(clk, rst, clk_enable, select, in, out);
parameter sync_type = "SYNC";
localparam synchronous = (sync_type == "SYNC");
localparam asynchronous = (sync_type == "ASYNC");
parameter WIDTH = 18;

input clk;
input rst;
input clk_enable;
input select;
input [WIDTH - 1 : 0] in;
output [WIDTH - 1 : 0] out;

reg [WIDTH - 1 : 0] d_ff;

```

```

generate
    if(synchronous) begin
        always@(posedge clk) begin
            if(rst) begin
                d_ff <= 0;
            end
            else if(clk_enable)begin
                d_ff <= in;
            end
        end
    end
end

else if(asynchronous) begin
    always@(posedge clk or posedge rst) begin
        if(rst) begin
            d_ff <= 0;
        end

        else if(clk_enable)begin
            d_ff <= in;
        end
    end
end

endgenerate

assign out = (select == 1) ? d_ff : in;

endmodule

```

Testbench code

```
module dsp48a1_tb();
  ///! 1. The parameters will remain unchanged.
  ///! Control Input Ports
  reg CLK;
  reg [7 : 0] OPMODE;
  ///! Reset Input Ports: All the resets are active high reset. They are either sync
  or async depending on the parameter RSTTYPE.
  reg RSTA, RSTB, RSTC, RSTCARRYIN, RSTD, RSTM, RSTOPMODE, RSTP;
  ///! Clock Enable Input Ports
  reg CEA, CEB, CEC, CECARRYIN, CED, CEM, CEOPMODE, CEP;
  /// ! Data Ports:
  reg [17 : 0] A, D, B, BCIN;
  reg CARRYIN;
  reg [47 : 0] PCIN, C;
  wire [17 : 0] BCOUT;
  wire [47 : 0] PCOUT, P;
  wire [35 : 0] M;
  wire CARRYOUT, CARRYOUTF;
  ///! out expected
  reg [17 : 0] BCOUT_expected;
  reg [47 : 0] PCOUT_expected, P_expected;
  reg [35 : 0] M_expected;
  reg CARRYOUT_expected, CARRYOUTF_expected;
  reg [47 : 0] past_value_of_P;
  reg past_value_of_CARRYOUT;
  reg [48 : 0] result;

  dsp48a1 uut(
    CLK, RSTA, RSTB, RSTC, RSTD, RSTM, RSTOPMODE, RSTP, RSTCARRYIN,
    CEA, CEB, CEC, CECARRYIN, CED, CEM, CEOPMODE, CEP, A, B, C, D,
    BCIN, PCIN, CARRYIN, OPMODE,
    CARRYOUT, CARRYOUTF, BCOUT, M, PCOUT, P
  );
  ///! clock generation
  initial begin
    CLK = 0;
    forever
      #1 CLK = ~CLK;
  end
end
```

```

//! 2. Stimulus Generation (Initial Block)
initial begin
    //! initialize input
    A = 0; B = 0; C = 0; D = 0; BCIN = 0; CARRYIN = 0;
    OPMODE = 0; CEA = 0; CEB = 0; CEC = 0; CECARRYIN = 0;
    CED = 0; CEM = 0; CEOPMODE = 0; CEP = 0; PCIN = 0;
    BCOUNT_expected = 0; PCOUT_expected = 0; M_expected = 0; P_expected = 0;
    CARRYOUT_expected = 0; CARRYOUTF_expected = 0;
    past_Value_of_P = 0; result = 0;

    //! 2.1. Verify Reset Operation
    //Assert all active-high reset signals by setting them to 1.
    RSTA = 1; RSTB = 1; RSTC = 1; RSTCARRYIN = 1;
    RSTD = 1; RSTM = 1; RSTOPMODE = 1; RSTP = 1;

    //Drive remaining inputs with arbitrary (random) values.
    A = $random; B = $random; C = $random; D = $random; BCIN = $random;
    CARRYIN = $random; OPMODE = $random; PCIN = $random;
    CEA = $random; CEB = $random; CEC = $random; CECARRYIN = $random;
    CED = $random; CEM = $random; CEOPMODE = $random; CEP = $random;

    //Wait for the negative edge of the clock.
    @(negedge CLK);

    //Add a condition for self-checking to verify that all outputs are zero.
    BCOUNT_expected = 0; PCOUT_expected = 0; M_expected = 0; P_expected = 0;
    CARRYOUT_expected = 0; CARRYOUTF_expected = 0;
    if(BCOUNT_expected != BCOUNT)
        $display("ERROR IN RESET BCOUNT");
    else
        $display("RESET BCOUNT pass");
    if(PCOUT_expected != PCOUT)
        $display("ERROR IN RESET PCOUT");
    else
        $display("RESET PCOUT pass");
    if(M_expected != M)
        $display("ERROR IN RESET M");
    else
        $display("RESET M pass");
    if(P_expected != P)
        $display("ERROR IN RESET P");
    else
        $display("RESET P pass");

```

```

if(CARRYOUT_expected != CARRYOUT)
    $display("ERROR IN RESET CARRYOUT");
else
    $display("RESET CARRYOUT pass");
if(CARRYOUTF_expected != CARRYOUTF)
    $display("ERROR IN RESET CARRYOUTF");
else
    $display("RESET CARRYOUTF pass");

//Deassert all reset signals and assert all clock enable signals to validate
the functionality of the subsequent DSP paths.
RSTA = 0; RSTB = 0; RSTC = 0; RSTCARRYIN = 0; RSTD = 0; RSTM = 0;
RSTOPMODE = 0; RSTP = 0; CEA = 1; CEB = 1; CEC = 1; CECARRYIN = 1;
CED = 1; CEM = 1; CEOPMODE = 1; CEP = 1;

//! 2.2. Verify DSP Path 1
//corresponding to OPMODE = 8'b11011101.
OPMODE = 8'b11011101;

// Apply the following input values: A = 20, B = 10, C = 350, and D = 25.
A = 20; B = 10; C = 350; D = 25;

//Drive BCIN, PCIN, and CARRYIN with arbitrary (random) values.
BCIN = $random; CARRYIN = $random; PCIN = $random;

//The expected outputs are: BCOUT = 'hf, M = 'h12c, P = PCOUT = 'h32, and
CARRYOUT = CARRYOUTF = 0.
BCOUT_expected = 'hf; PCOUT_expected = 'h32; M_expected = 'h12c;
P_expected = 'h32; CARRYOUT_expected = 0; CARRYOUTF_expected = 0;

//Wait for four negative clock edges, as the data propagates through four
flip-flops (DREG, B1REG, MREG, and PREG)
repeat(4) @(negedge CLK);

//Add a condition for self-checking to verify that the design outputs with
the expected outputs.
if(BCOUT_expected != BCOUT)
    $display("ERROR IN Path 1 BCOUNT");
else
    $display("Path 1 BCOUNT pass");
if(PCOUT_expected != PCOUT)
    $display("ERROR IN Path 1 PCOUT");
else
    $display("Path 1 PCOUT pass");

```

```

if(M_expected != M)
    $display("ERROR IN Path 1 M");
else
    $display("Path 1 M pass");
if(P_expected != P)
    $display("ERROR IN Path 1 P");
else
    $display("Path 1 P pass");
if(CARRYOUT_expected != CARRYOUT)
    $display("ERROR IN Path 1 CARRYOUT");
else
    $display("Path 1 CARRYOUT pass");
if(CARRYOUTF_expected != CARRYOUTF)
    $display("ERROR IN Path 1 CARRYOUTF");
else
    $display("Path 1 CARRYOUTF pass");

//! 2.3. Verify DSP Path 2
// corresponding to OPMODE = 8'b00010000.
OPMODE = 8'b00010000;

//Apply the following input values: A = 20, B = 10, C = 350, and D = 25.
A = 20; B = 10; C = 350; D = 25;

//Drive BCIN, PCIN, and CARRYIN with arbitrary (random) values.
BCIN = $random; CARRYIN = $random; PCIN = $random;

// The expected outputs are: BCOUT = 'h23, M = 'h2bc, P = PCOUT = 0, and
CARRYOUT = CARRYOUTF = 0.
BCOUT_expected = 'h23; PCOUT_expected = 'h0; M_expected = 'h2bc;
P_expected = 'h0; CARRYOUT_expected = 0; CARRYOUTF_expected = 0;

//Wait for three negative edges
repeat(3) @(negedge CLK);

//Add a condition for self-checking to verify that the design outputs with
the expected outputs.
if(BCOUT_expected != BCOUT)
    $display("ERROR IN Path 2 BCOUNT");
else
    $display("Path 2 BCOUNT pass");
if(PCOUT_expected != PCOUT)
    $display("ERROR IN Path 2 PCOUT");
else
    $display("Path 2 PCOUT pass");

```

```

if(M_expected != M)
    $display("ERROR IN Path 2 M");
else
    $display("Path 2 M pass");
if(P_expected != P)
    $display("ERROR IN Path 2 P");
else
    $display("Path 2 P pass");
if(CARRYOUT_expected != CARRYOUT)
    $display("ERROR IN Path 2 CARRYOUT");
else
    $display("Path 2 CARRYOUT pass");
if(CARRYOUTF_expected != CARRYOUTF)
    $display("ERROR IN Path 2 CARRYOUTF");
else
    $display("Path 2 CARRYOUTF pass");

//! 2.4. Verify DSP Path 3
//corresponding to OPMODE = 8'b00001010
past_Value_of_P = P;
past_value_of_CARRYOUT = CARRYOUT;
OPMODE = 8'b00001010;

//Apply the following input values: A = 20, B = 10, C = 350, and D = 25.
A = 20; B = 10; C = 350; D = 25;

//Drive BCIN, PCIN, and CARRYIN with arbitrary (random) values.
BCIN = $random; CARRYIN = $random; PCIN = $random;

//The expected outputs are: BCOUT = 'ha, M = 'hc8, P = PCOUT which is the
past Value of P, and CARRYOUT = CARRYOUTF which is the past value of CARRYOUT.
BCOUT_expected = 'ha;
result = 2 * past_Value_of_P;
PCOUT_expected = result[47 : 0];
M_expected = 'hc8;
P_expected = result[47 : 0];
CARRYOUT_expected = result[48];
CARRYOUTF_expected = result[48];

//Wait for three negative edges
repeat(3) @(negedge CLK);

```

```

//Add a condition for self-checking to verify that the design outputs with
the expected outputs.
if(BCOUT_expected != BCOUT)
    $display("ERROR IN Path 3 BCOUNT");
else
    $display("Path 3 BCOUNT pass");
if(PCOUT_expected != PCOUT)
    $display("ERROR IN Path 3 PCOUT");
else
    $display("Path 3 PCOUT pass");
if(M_expected != M)
    $display("ERROR IN Path 3 M");
else
    $display("Path 3 M pass");
if(P_expected != P)
    $display("ERROR IN Path 3 P");
else
    $display("Path 3 P pass");
if(CARRYOUT_expected != CARRYOUT)
    $display("ERROR IN Path 3 CARRYOUT");
else
    $display("Path 3 CARRYOUT pass");
if(CARRYOUTF_expected != CARRYOUTF)
    $display("ERROR IN Path 3 CARRYOUTF");
else
    $display("Path 3 CARRYOUTF pass");

//! 2.5. Verify DSP Path 4
//corresponding to OPMODE = 8'b10100111.
OPMODE = 8'b10100111;

//Apply the following input values: A=5, B=6, C=350, D=25 and PCIN=3000
A = 5; B = 6; C = 350; D = 25; PCIN = 3000;

// Drive BCIN, and CARRYIN with arbitrary (random) values.
BCIN = $random; CARRYIN = $random;

//The expected outputs are: BCOUT = 'h6, M = 'h1e,
P = PCOUT = 'hfe6fffec0bb1, and CARRYOUT = CARRYOUTF = 1.
BCOUT_expected = 'h6; PCOUT_expected = 'hfe6fffec0bb1; M_expected = 'h1e;
P_expected = 'hfe6fffec0bb1; CARRYOUT_expected = 1; CARRYOUTF_expected = 1;
//Wait for three negative edges
repeat(3) @(negedge CLK);

```

```

//Add a condition for self-checking to verify that the design outputs with
the expected outputs.
if(BCOUT_expected != BCOUT)
    $display("ERROR IN Path 4 BCOUNT");
else
    $display("Path 4 BCOUNT pass");
if(PCOUT_expected != PCOUT)
    $display("ERROR IN Path 4 PCOUT");
else
    $display("Path 4 PCOUT pass");
if(M_expected != M)
    $display("ERROR IN Path 4 M");
else
    $display("Path 4 M pass");
if(P_expected != P)
    $display("ERROR IN Path 4 P");
else
    $display("Path 4 P pass");
if(CARRYOUT_expected != CARRYOUT)
    $display("ERROR IN Path 4 CARRYOUT");
else
    $display("Path 4 CARRYOUT pass");
if(CARRYOUTF_expected != CARRYOUTF)
    $display("ERROR IN Path 4 CARRYOUTF");
else
    $display("Path 4 CARRYOUTF pass");
$stop;
end
endmodule

```

Do file

```

1 vlib work
2 vlog DSP48A1.v DSP48A1_tb.v
3 vsim -voptargs=+acc work.dsp48a1_tb
4 add wave *
5 run -all
6 #quit -sim

```

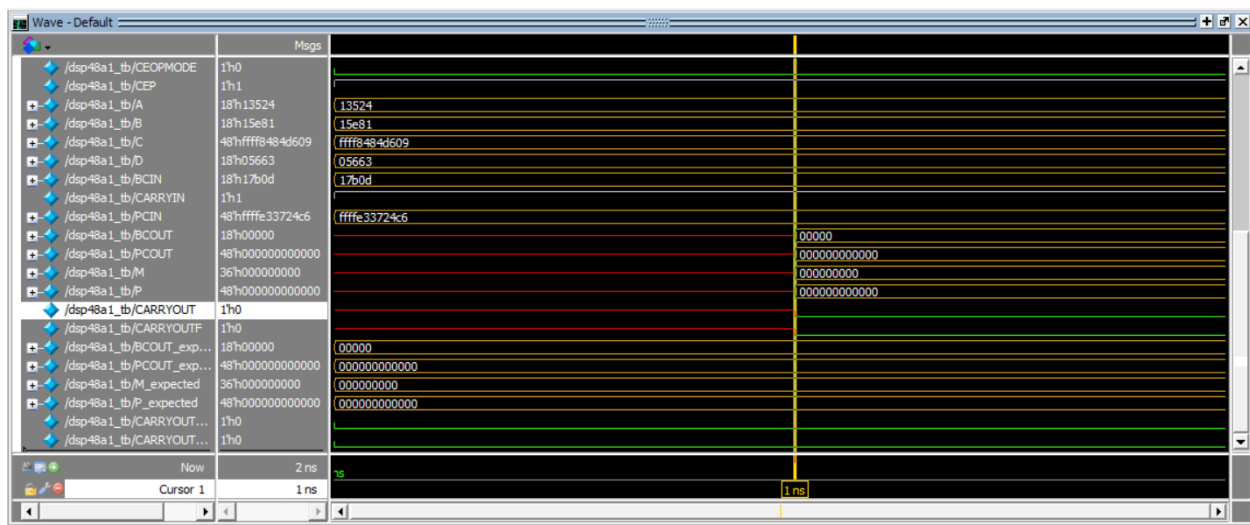
QuestaSim Snippets

Verify Reset Operation

Verify Reset Operation

- Assert all active-high reset signals by setting them to 1.
- Drive remaining inputs with arbitrary (random) values.
- Wait for the negative edge of the clock.
- Add a condition for self-checking to verify that all outputs are zero.
- De assert all reset signals and assert all clock enable signals to validate the functionality of the subsequent DSP paths.

- Waveform



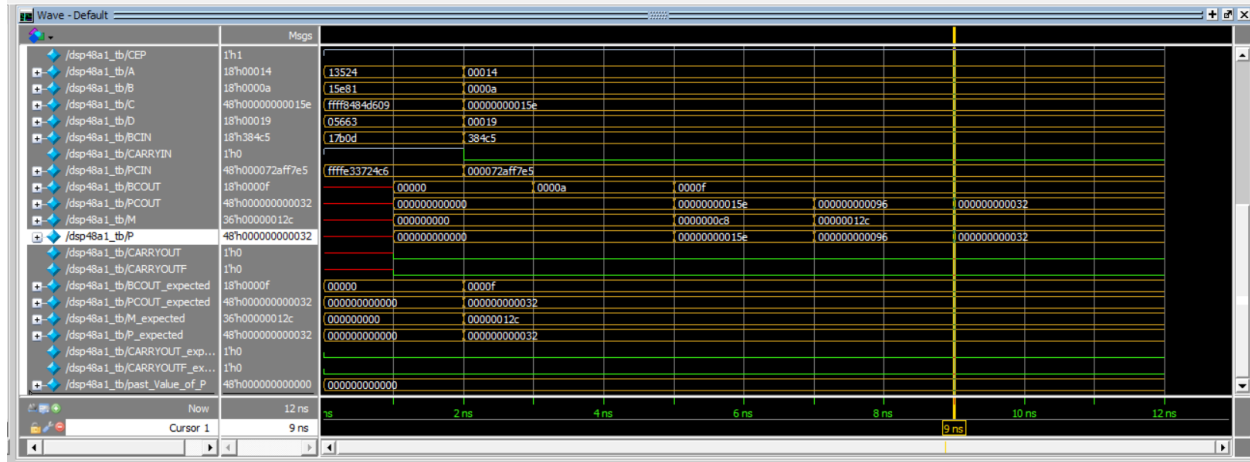
- Transcript

```
Transcript
# Loading work.REG_MUX(fast__4)
# RESET BCOUNT pass
# RESET PCOUT pass
# RESET M pass
# RESET P pass
# RESET CARRYOUT pass
# RESET CARRYOUTF pass
# ** Note: $stop : DSP48A1_tb.v(287)
# Time: 2 ns Iteration: 1 Instance: /dsp48a1_tb
# Break in Module dsp48a1_tb at DSP48A1_tb.v line 287

0 ns to 2 ns Project : project_dsp Now: 2 ns Delta: 1 /dsp48a1_tb/M_expected
```

Figure 1 Path 1 Data Flow

- Waveform path 1



- Transcript path 1

```

Transcript
# Path 1 BCOUNT pass
# Path 1 PCOUT pass
# Path 1 M pass
# Path 1 P pass
# Path 1 CARRYOUT pass
# Path 1 CARRYOUTF pass
# ** Note: $stop : DSP48A1_tb.v(287)
#   Time: 12 ns Iteration: 1 Instance: /dsp48a1_tb
# Break in Module dsp48a1_tb at DSP48A1_tb.v line 287
0 ns to 13 ns    Project: project_dsp    Now: 12 ns Delta: 1    /dsp48a1_tb/CEOPMODE

```

Verify DSP Path 2

Verify DSP Path 2

- This test path evaluates the pre-addition allowing it to propagate and post-addition stages by enabling the zeros to route through Mux X and Mux Z, corresponding to OPMODE = 8'b00010000.
- Apply the following input values:
A = 20, B = 10, C = 350, and D = 25.
- Drive BCIN, PCIN, and CARRYIN with arbitrary (random) values.
- Wait for three negative edges, as the data propagates through three flip-flops (DREG, B1REG, MREG) as longest path and also pass to PREG in parallel, as illustrated in Figure (2).
- The expected outputs are: BCOUT = 'h23, M = 'h2bc, P = PCOUT = 0, and CARRYOUT = CARRYOUTF = 0.
- Add a condition for self-checking to verify that the design outputs with the expected outputs.

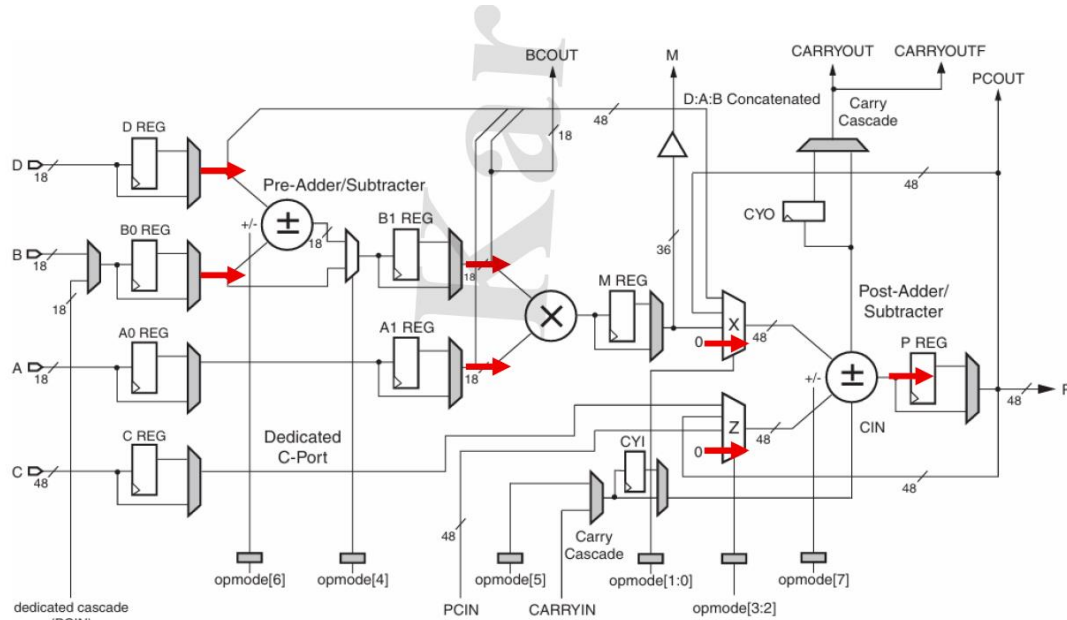
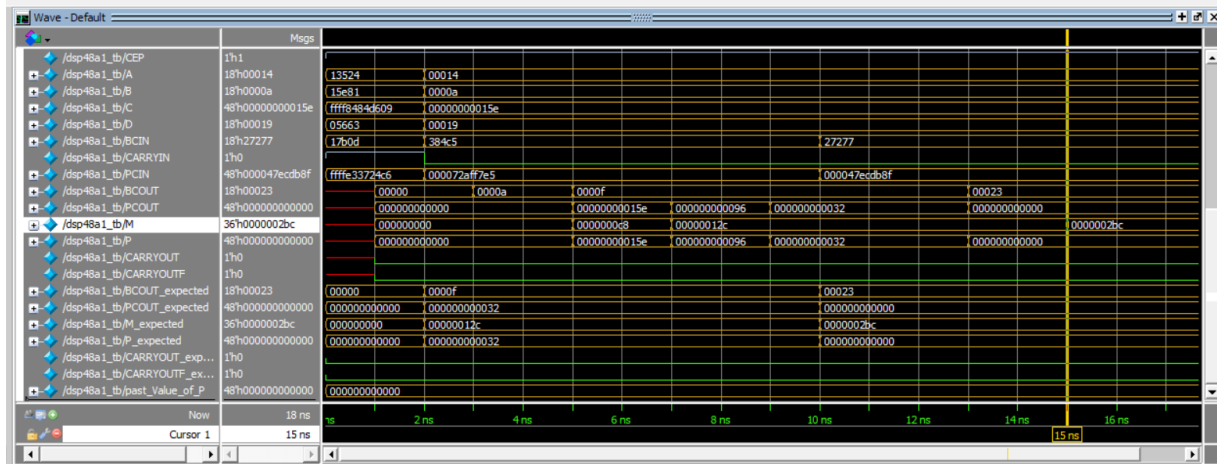


Figure 2 Path 2 Data Flow

- Waveform path 2



- Transcript path 2

```

Transcript
# Path 1 CARRYOUTF pass
# Path 2 BCOUNT pass
# Path 2 PCOUT pass
# Path 2 M pass
# Path 2 P pass
# Path 2 CARRYOUT pass
# Path 2 CARRYOUTF pass
# ** Note: $stop : DSP48A1_tb.v(360)
# Time: 18 ns Iteration: 1 Instance: /dsp48a1_tb
# Break in Module dsp48a1_tb at DSP48A1_tb.v line 360

0 ns to 19 ns    Project : project_dsp    Now: 18 ns Delta: 1    sim:/dsp48a1_tb/#INITIAL#83

```

Verify DSP Path 3

Verify DSP Path 3

- This test path has no pre-addition/subtractor (doesn't allow it to propagate) and allows post-addition stages by enabling the P feedback to route through Mux X and Mux Z, corresponding to $OPMODE = 8'b00001010$.
- Apply the following input values:
 $A = 20$, $B = 10$, $C = 350$, and $D = 25$.
- Drive BCIN, PCIN, and CARRYIN with arbitrary (random) values.
- Wait for three negative edges, as the data propagates through three flip-flops (B1REG, MREG, PREG) as longest path and also pass to PREG in parallel, as illustrated in Figure (3).
- The expected outputs are: $BCOUT = 'ha$, $M = 'hc8$, $P = PCOUT$ which is the past Value of P, and $CARRYOUT = CARRYOUTF$ which is the past value of CARRYOUT.
- Add a condition for self-checking to verify that the design outputs with the expected outputs.

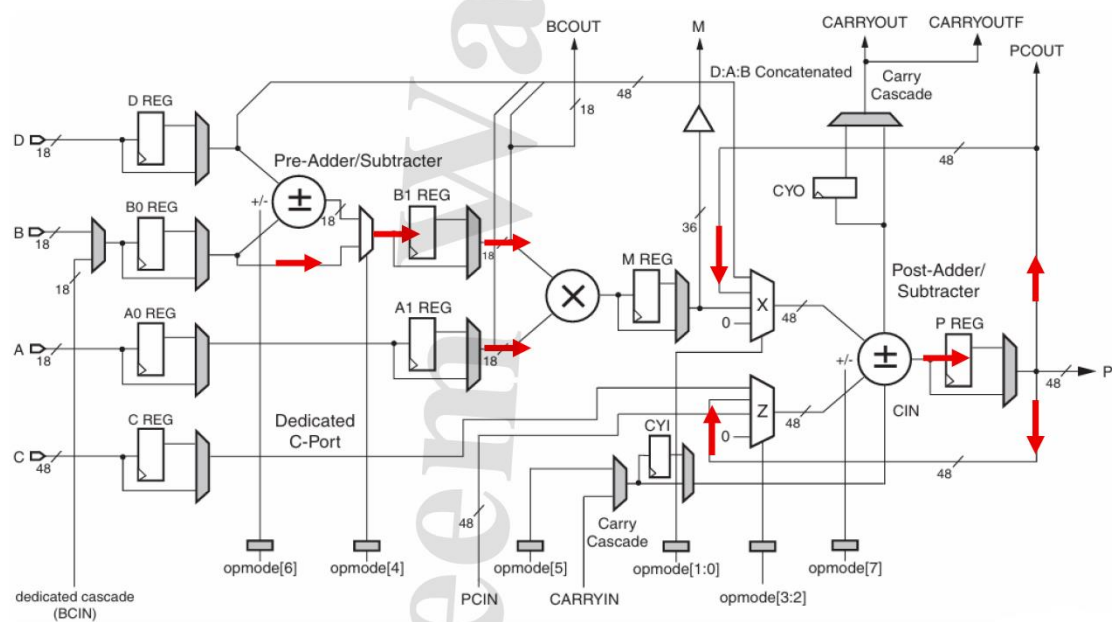
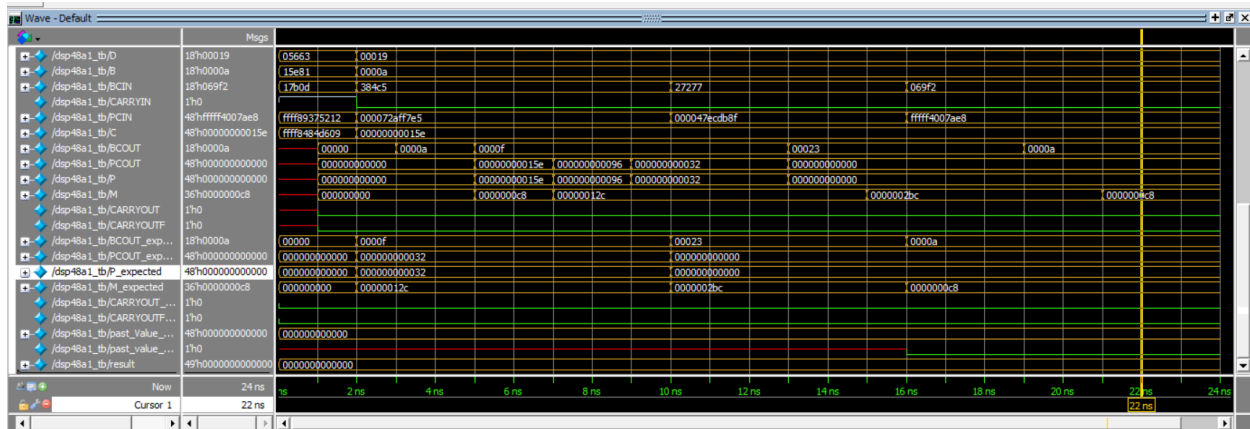


Figure 3 Path 3 Data Flow

- Waveform path 3



- Transcript path 3

```

Transcript
# Path 2 CARRYOUTF pass
# Path 3 BCOUNT pass
# Path 3 PCOUT pass
# Path 3 M pass
# Path 3 P pass
# Path 3 CARRYOUT pass
# Path 3 CARRYOUTF pass
# ** Note: $stop : DSP48A1_tb.v(243)
# Time: 24 ns Iteration: 1 Instance: /dsp48a1_tb
# Break in Module dsp48a1_tb at DSP48A1_tb.v line 243

0 ns to 24 ns | Project : project_dsp | Now: 24 ns Delta: 1 | /dsp48a1_tb/P_expected

```

Verify DSP Path 4

Verify DSP Path 4

- This test path has no pre-addition/subtractor (doesn't allow it to propagate) and allows post-subtractor stages by enabling D:A:B Concatenated to route through Mux X and PCIN to Mux Z, corresponding to OPMODE = 8'b10100111.
- Apply the following input values:
A = 5, B = 6, C = 350, D = 25 and PCIN = 3000
- Drive BCIN, and CARRYIN with arbitrary (random) values.
- Wait for three negative edges, as the data propagates through three flip-flops (B1REG, MREG, PREG) as longest path as illustrated in Figure (4).
- The expected outputs are: BCOUT = 'h6, M = 'h1e, P = PCOUT = 'hfe6fffec0bb1, and CARRYOUT = CARRYOUTF = 1.
- Add a condition for self-checking to verify that the design outputs with the expected outputs.

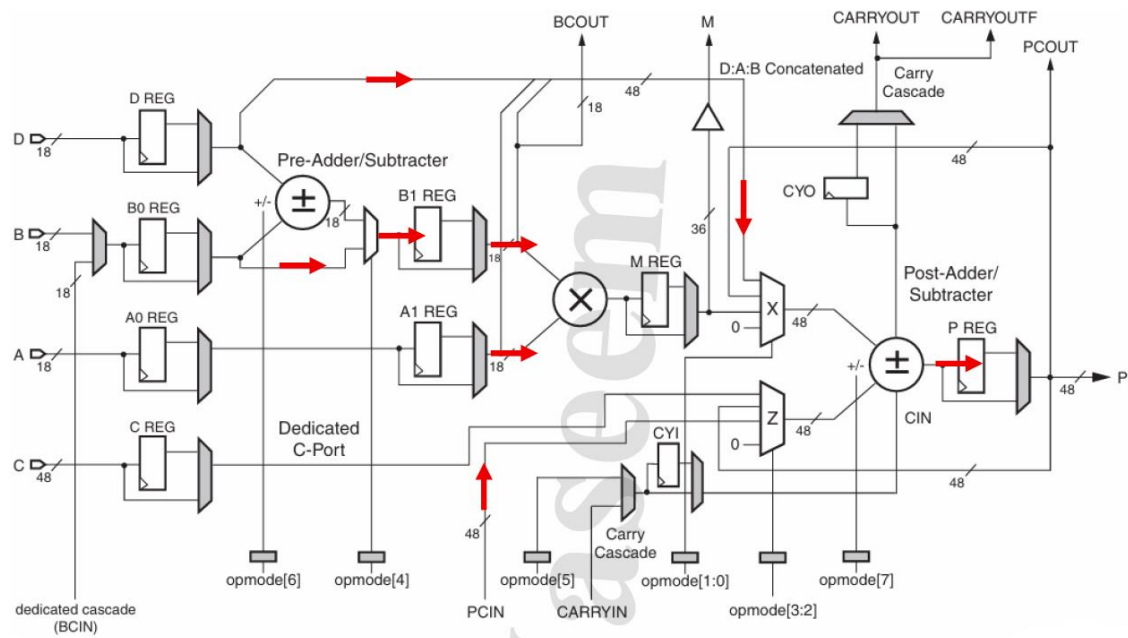
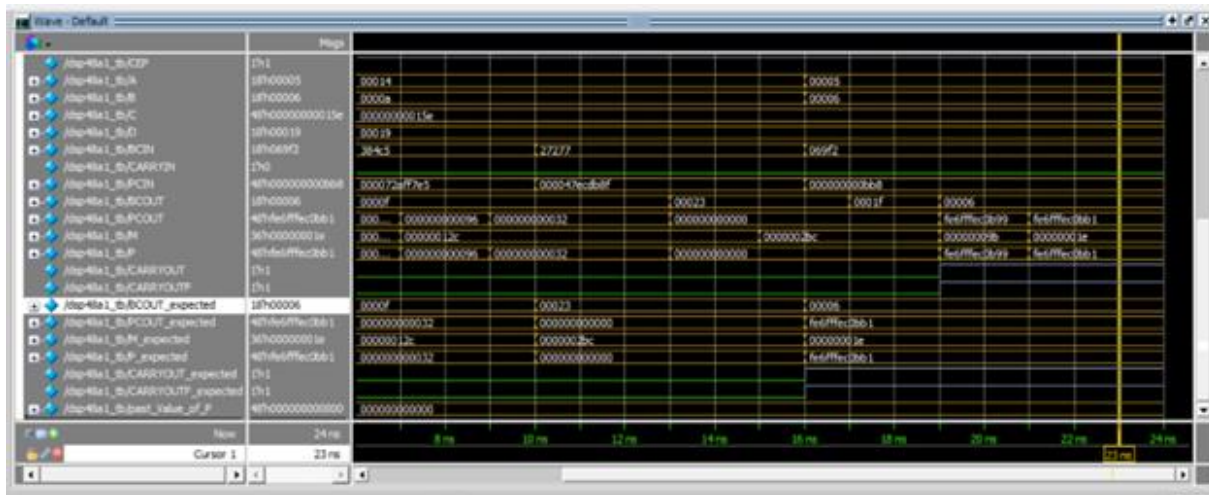


Figure 4 Path 4 Data Flow

- Waveform path 4



- Transcript path 4

```

Transcript
# Path 4 BCOUNT pass
# Path 4 PCOUT pass
# Path 4 M pass
# Path 4 P pass
# Path 4 CARRYOUT pass
# Path 4 CARRYOUTF pass
# ** Note: $stop : DSP48A1_tb.v(521)
# Time: 24 ns Iteration: 1 Instance: /dsp48a1_tb
# Break in Module dsp48a1_tb at DSP48A1_tb.v line 521

6 ns to 25 ns      Project: project_dsp      Now: 24 ns Delta: 1      sim:/dsp48a1_tb/#INITIAL#84

```

Constraint File

```
## Clock signal
set_property -dict { PACKAGE_PIN W5    IOSTANDARD LVCMOS33 } [get_ports CLK]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports CLK]


## Configuration options, can be used for all designs
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property CFGBVS VCC0 [current_design]

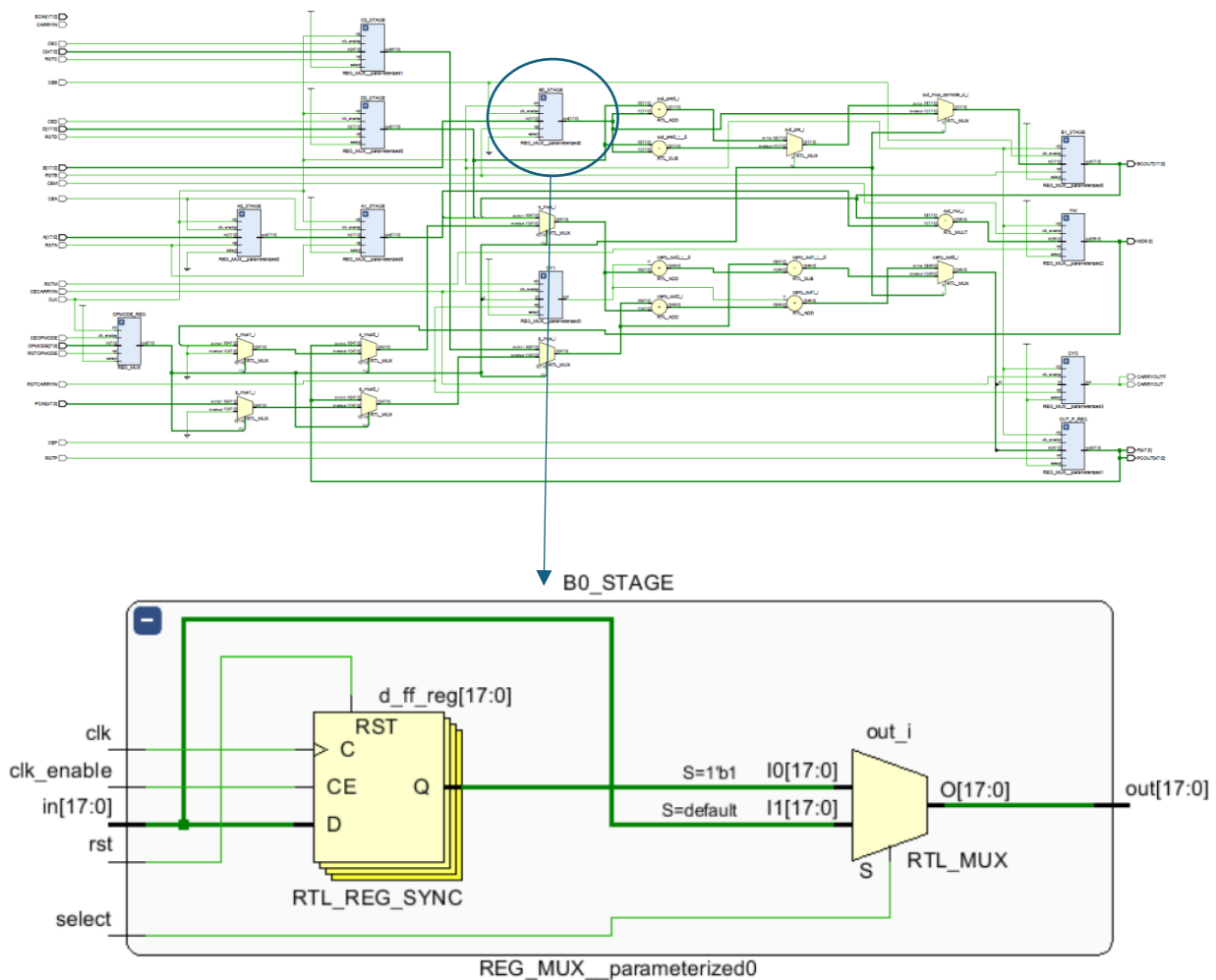

## SPI configuration mode options for QSPI boot, can be used for all designs
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 33 [current_design]
set_property CONFIG_MODE SPIx4 [current_design]
```

Elaboration

- Messages



- Schematic snippets



Synthesis

- Messages

Tcl ConsoleMessages xLogReportsDesign RunsTimingDebug

Warning (80)

Info (350)

Status (695)

Show All

[Project 1-111] Unisim Transformation Summary:
No Unisim elements were transformed.

Synthesis (57 warnings, 48 Infos)

[Common 17-349] Got license for feature 'Synthesis' and/or device 'xc7a200t'

[Synth 8-6157] synthesizing module 'dsp48a1' [DSP48A1.v:1] (5 more like this)

[Synth 8-6155] done synthesizing module 'REG_MUX' (1#1) [DSP48A1.v:151] (5 more like this)

[Synth 8-3331] design dsp48a1 has unconnected port BCIN[17] (37 more like this)

[Device 21-403] Loading part xc7a200tfg1156-3

[Project 1-236] Implementation specific constraints were found while reading constraint file [D:/DIGITAL/projects/Spartan6_DSP48A1/dsp48a1_vivado/project_2/dsp_constraints.xdc]. These constraints will be ignored for synthesis but will be used in implementation. Impacted constraints are listed in the file [Xil/dsp48a1_propimpl.xdc]. Resolution: To avoid this warning, move constraints listed in [Undefined] to another XDC file and exclude this new file from synthesis with the used_in_synthesis property (File Properties dialog in GUI) and re-run elaboration/synthesis.

[Synth 8-5818] HDL ADVISOR - The operator resource <adder> is shared. To prevent sharing consider applying a KEEP on the output of the operator [DSP48A1.v:101] (1 more like this)

- Utilization report

ReportsDesign RunsUtilization xTimingDebug

Hierarchy

Name	Slice LUTs (134600)	Slice Registers (269200)	DSP s (740)	Bonded IOB (500)	BUFGCTRL (32)
▼ N dsp48a1	230	160	1	327	1
A1_STAGE (REG_MUX...	0	18	0	0	0
B1_STAGE (REG_MUX...	0	18	0	0	0
C0_STAGE (REG_MU...	0	48	0	0	0
CY1 (REG_MUX_par...	1	1	0	0	0
CYO (REG_MUX_par...	0	1	0	0	0
D0_STAGE (REG_MU...	0	18	0	0	0
OPMODE_REG (REG_...	228	8	0	0	0
OUT_P_REG (REG_M...	0	48	0	0	0

- Timing report

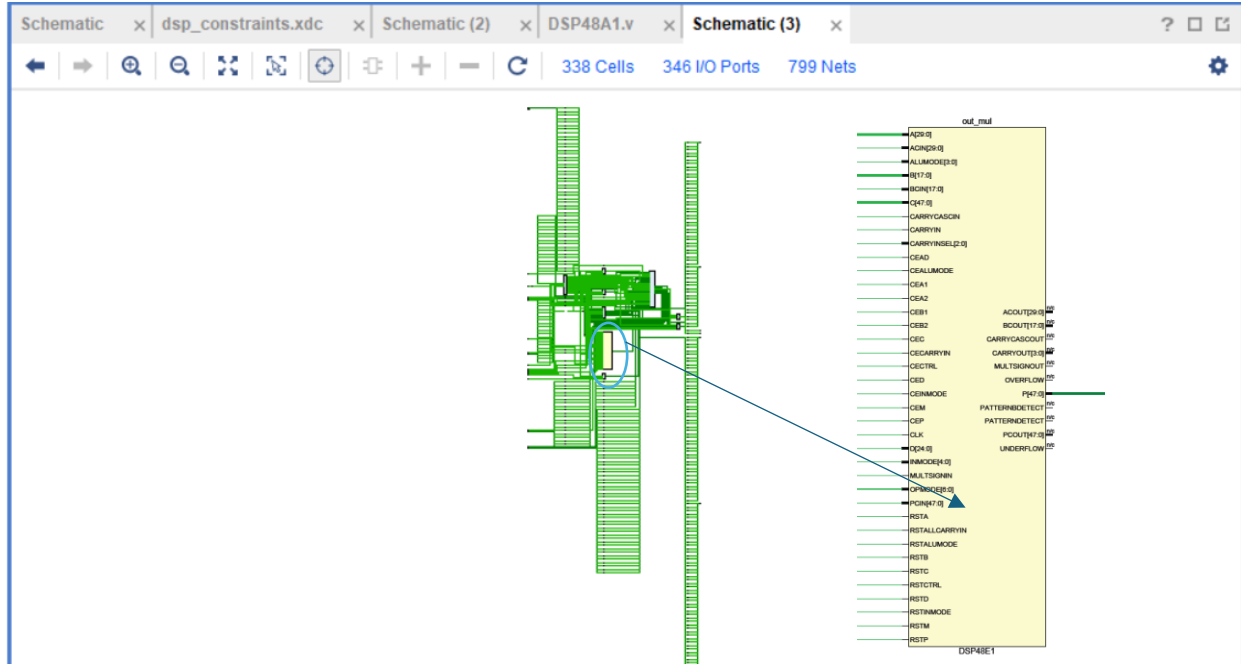
Design RunsUtilizationTiming xDebug

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.168 ns	Worst Hold Slack (WHS): 0.182 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 106	Total Number of Endpoints: 106	Total Number of Endpoints: 162

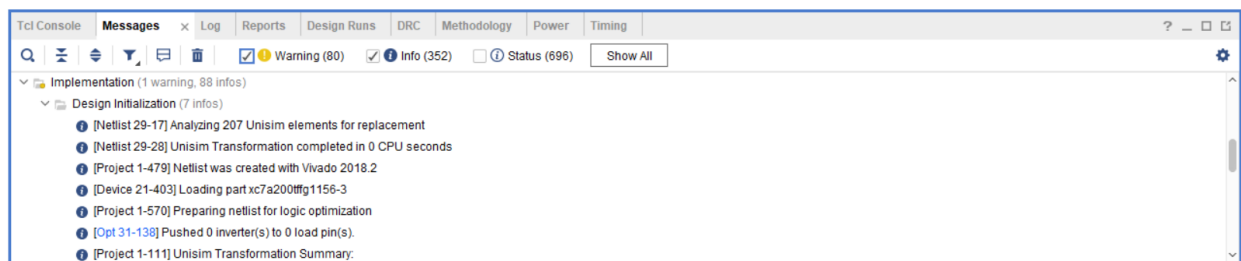
All user specified timing constraints are met.

- Schematic snippets



Implementation

- Messages



- Utilization report

Reports

Design Runs

DRC

Methodology

Power

Timing

Utilization

Q

≡

⌵

%

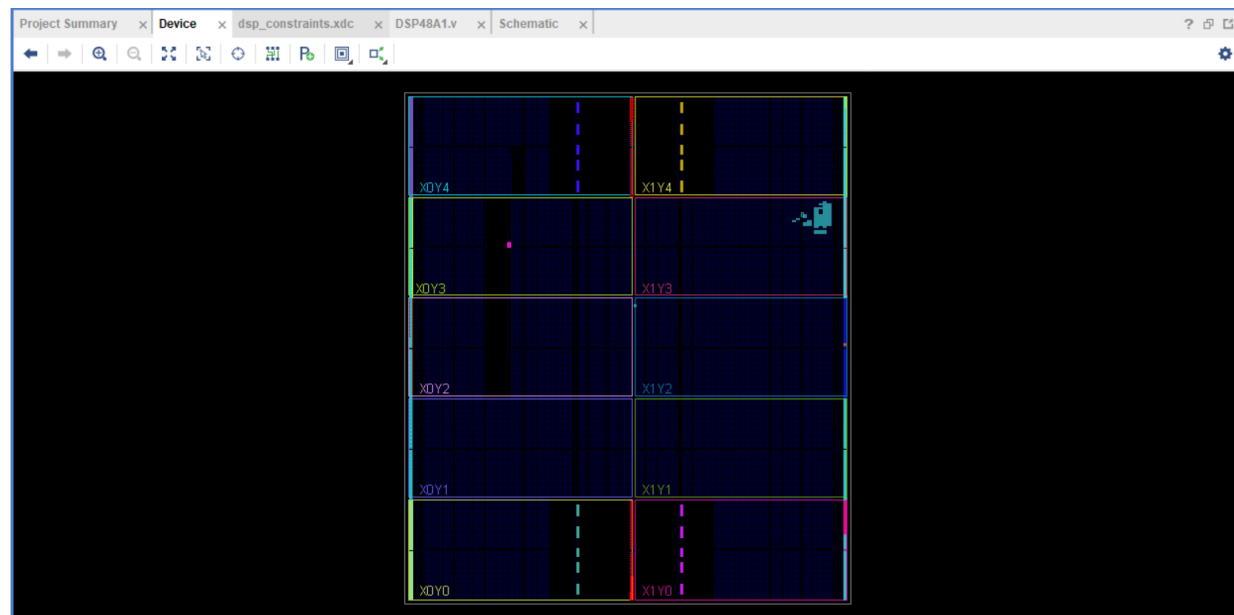
Hierarchy

Name	Slice LUTs (133800)	Slice Registers (267600)	Slice (33450)	LUT as Logic (133800)	LUT Flip Flop Pairs (133800)	DSPs (740)	Bonded IOB (500)	BUFGCTRL (32)	
▼ N dsp48a1	229	179	100	229	50	1	327	1	
A1_STAGE (REG_MUX...	0	18	6	0	0	0	0	0	
B1_STAGE (REG_MUX...	0	36	12	0	0	0	0	0	
C0_STAGE (REG_MU...	0	48	14	0	0	0	0	0	
CY1 (REG_MUX__par...	1	1	1	1	1	0	0	0	
CY0 (REG_MUX__par...	0	2	2	0	0	0	0	0	
D0_STAGE (REG_MU...	0	18	7	0	0	0	0	0	
OPMODE_REG (REG_...	228	8	73	228	0	0	0	0	
OUT_P_REG (REG_M...	0	48	12	0	0	0	0	0	

- Timing report

s	Design Runs	DRC	Methodology	Power	Timing	x	Utilization				?	_	□	↻
Design Timing Summary														
Setup					Hold					Pulse Width				
Worst Negative Slack (WNS): 3.932 ns					Worst Hold Slack (WHS): 0.273 ns					Worst Pulse Width Slack (WPWS): 4.500 ns				
Total Negative Slack (TNS): 0.000 ns					Total Hold Slack (THS): 0.000 ns					Total Pulse Width Negative Slack (TPWS): 0.000 ns				
Number of Failing Endpoints: 0					Number of Failing Endpoints: 0					Number of Failing Endpoints: 0				
Total Number of Endpoints: 125					Total Number of Endpoints: 125					Total Number of Endpoints: 181				
All user specified timing constraints are met.														

- Device snippets



Linting

- Snippets showing no errors

The screenshot shows a Verilog code editor with a file named `DSP48A1.v` open. The code defines a module `dsp48a1` with various input and output signals. It includes a parameter block for `A0REG`, `A1REG`, `B0REG`, and `B1REG`, each with a default value of `1'b0` or `1'b1`. Comments explain that these parameters define the number of pipeline registers and stages.

On the right side, a **Lint Summary** window is open, displaying a table of lint results. The table has two columns: **Name** and **Count**. The results are as follows:

Name	Count
Open(uninspected, pendi...	8
Warning	5
mux_select_const	5
Info	3